REVIO STOCKSENSE

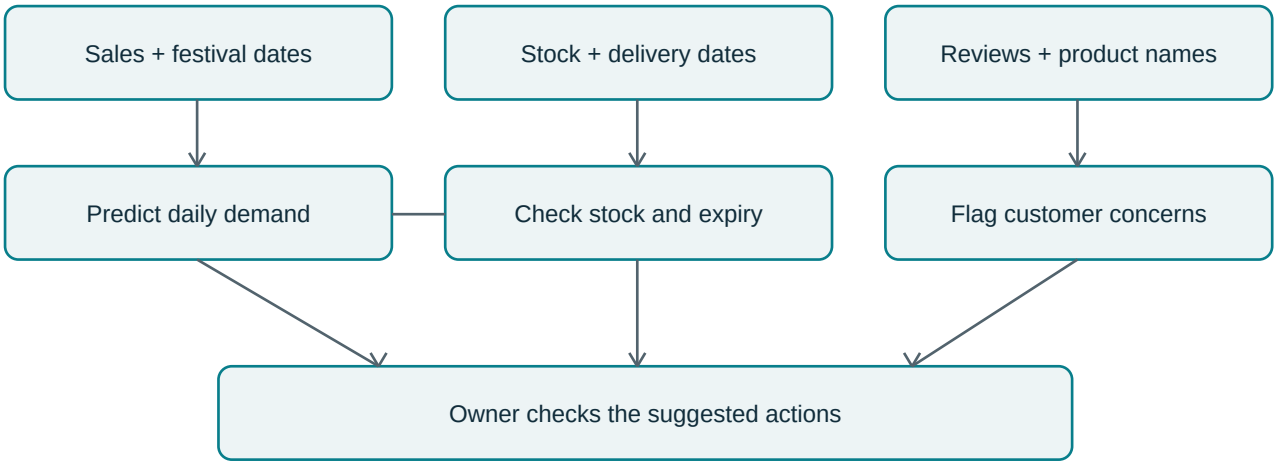
Complete Project Explanation

A simple guide to the screens, files and key terms, with 40 short interview answers.

17 October 2026
Independent proposal for Cynbit AI/ML Engineer Internship

The shop data is generated by a program. Real-shop accuracy and savings are not yet known. The app does not connect to Revio or place orders.

System at a glance



A plain-English guide to the model, its errors, the dataset, the screens and every main project file. All sales results use computer-generated data.

Part I - Understand the project

Start here

StockSense is a working demo for a shop owner. It estimates future demand and helps the owner review stock purchases, expiry risks and customer requests. It does not place orders. Its sales data is made up by a program, so it cannot yet tell us how well it will work in a real shop.

Read the AI Opportunity Report for the business idea. Read the Technical Approach Report for how the calculations work. This guide explains the screens, files and words you will see while using or presenting the project.

Which model does the app use?

The current demo uses **Seasonal Poisson Regression**. The Python class is called `PoissonRegressor`, from the scikit-learn library. A separate model is trained for each product that has enough sales history.

A regression model predicts a number. Here, that number is the expected units sold per day. Poisson regression is designed for counts and keeps the expected number above zero. A result such as 7.5 means an average expectation, not that the shop can sell half a packet.

The model learns four kinds of patterns together: changes over the year, differences between weekdays, slow growth or decline over time, and festival preparation days. These produce **one forecast**, which the ordering and expiry pages also use.

The annual pattern uses sine and cosine waves. These are smooth repeating curves that help the model move gradually from one season to the next. The model does not simply apply a rule such as "increase every product in winter."

Festival preparation means the seven days before a festival plus the festival day. For each product, the current model learns one shared festival effect from all past festival windows. It does not learn a separate Diwali effect and Holi effect. We made this choice because there are only two or three past examples of each named festival. Separate effects were reacting too strongly to random variation.

Why choose this model?

It fits the kind of number we want to predict, can learn a different seasonal pattern for each product, and runs on a normal laptop. More importantly, it had the lowest average error in the earlier periods used to choose the model. This is a reason to use it in this demo, not proof that it is the best model for a real shop.

The app also compares it with three other methods. A 28-day average repeats the recent average. A weekly repeat method copies the last week's pattern. HistGradientBoosting is a machine-learning model that combines many small decision trees to learn more complex relationships.

When does the app avoid this model?

The seasonal model needs at least 180 usable daily observations spread across at least 365 days. Otherwise, it uses the 28-day average and labels that choice. If even recent sales history is too limited, it gives no exact forecast. A new product does not get a made-up demand number just because a review asks for it.

How accurate is it?

Real-shop accuracy is not known. The following numbers come from the project's computer-generated data. We should not present them as results from Cynbit customers or real shops.

Method	Error used to choose the model	Error in later test periods
Seasonal Poisson Regression	3.088 units/day	2.860 units/day
HistGradientBoosting	3.178 units/day	2.964 units/day
28-day average	3.507 units/day	3.374 units/day
Repeat the last week	4.230 units/day	4.133 units/day

Both error columns use **MAE: Mean Absolute Error**. It is the average size of a mistake, ignoring whether the forecast was too high or too low. If the actual sale is 10 and the forecast is 7, the error is 3 units. An MAE of 2.860 means the forecast was wrong by about 2.86 units per evaluated product per day on average. It does not mean every forecast is within that amount.

Other result	Value	What it means
WAPE	18.520%	Add all absolute errors, then divide by all actual units sold. Lower is better.
RMSE	3.921 units	An error score that gives extra weight to large mistakes. Lower is better.

Other result	Value	What it means
Bias	-0.040 units	Average forecast minus actual sales. The small negative value means slight overall underprediction.

Do not call this 81.48% accuracy. Subtracting WAPE from 100 does not create a reliable "percent correct" score. Forecasting predicts quantities, so the size of each error matters.

On the same current dataset, the chosen model's MAE is about 3.5% lower than boosting and 15.2% lower than the average method. That is a comparison of forecast errors, not a percentage of money saved. A three-unit error may be small for a product selling 50 units a day but serious for one selling four.

How were these numbers measured?

First, the models learned from older records and predicted four later 30-day periods. Those periods were used to choose the model. Next, the chosen method was kept fixed while all methods were tested over twelve more 30-day periods, covering 360 days.

At each step, training used only records from before the forecast date. As time moved forward, training could include records that had become available since the previous step. This imitates updating a model as a shop collects more sales.

The main test contains **4,200 product-days**. One product-day means one product on one date. There were 4,320 possible records for the 12 products across 360 days; 120 were left out because sales were limited by stock availability. The thirteenth product has no sales history and is not scored.

This project has been developed and adjusted using the same simulation. It is not an independent real-world test. Earlier versions also used different data, so their scores should not be used to claim a clean before-and-after improvement.

What is in the dataset?

The shop records are **synthetic**, which means a Python program created them. They are not downloaded from a real store and are not Cynbit data. The festival dates are the exception: they were imported from the public Google Calendar called Holidays in India.

Item	Current dataset specification
Store coverage	One fictional kirana store
Sales dates	1 January 2024 to 16 October 2026
History length	1,020 days for each of 12 products
Products	13 listed products; 12 have sales and one is a new product
Sales records	12,240 rows: one product on one day per row
Stock batches	24 rows describing stock held at the latest snapshot
Purchase orders	3 examples of incoming or planned supply

Item	Current dataset specification
Customer reviews	14 written examples, not real customer messages
Festival dates	28 entries: seven festival names across 2024-2027
Calendar import date	17 October 2026; saved copy, not automatic live sync
Dataset version	winter-calendar-v3
Random seed	42; this makes the same generator repeatable

Products include cola, water, mango drink, milk, yogurt, chips, tea, coffee, biscuits, chocolate, packaged sweets and rice. Sugar-free cola is the new product without sales history. Prices and stock values are examples in Indian rupees.

The generator combines a starting demand level with seasonal changes, weekends, gradual trends, festival periods, example promotions and random variation. It also creates supply interruptions so some days run out of stock. It deliberately includes expiry and overstock examples so those screens can be demonstrated.

The current tea and coffee example has a winter peak. The earlier demo curve peaked near October by mistake; the generator was corrected. That change is an assumption in a fictional example, not something discovered in real shop records. The gifting example also has a festival increase built into the generated demand. A model matching these patterns does not prove that it will match a real shop.

Why this data is a major limitation

Real demand can change with location, prices, promotions, weather, competitors, customer habits and supply problems. The small simulation does not capture all of that. A good-looking test result can therefore coexist with forecasts that feel unrealistic.

We need real sales and stock records, with permission to use them, before judging business usefulness. Those records should include dates, product quantities, price changes, promotions and stock availability. We should test future periods that were not used while adjusting the model, and inspect errors for each product, not only one average score.

What each screen does

Screen	What you use it for
Executive Dashboard	Start here. See the products needing attention, draft orders, sales history and upcoming festivals.
Product Demand Forecasting	Choose a product and view its next 7, 14 or 30 days. Seasons and festivals are part of this one forecast.
Smart Reorder Planner	See the suggested purchase quantity and the stock, delivery or supplier rule behind it.
Expiry Management	Check recorded batch dates and estimated units that may remain unsold before expiry.
Slow-Moving Inventory	Find products with low sales or more stock than the next 30 days appear to need.
Review Intelligence	Read product requests and availability or quality concerns found by simple English rules.
Data Management	Inspect templates, upload a complete dataset, manage calendar dates, run evaluation and save or load locally.

The review-period control means days between stock checks. The safety-buffer control means extra demand-days of stock. For example, a 1.5-day buffer and demand of 20 units a day adds 30 units. Both are settings for the owner's policy, not values proved correct by the model.

How to read the forecast chart

The solid grey line is recorded sales. The dashed line is the model's prediction. Shaded dates cover the week before a festival and the festival day. There is only one future forecast line. The monthly history chart inside the expandable section describes old sales; it is not another prediction.

A forecast can drop after a festival window ends. That is different from predicting a drop during the festival. Weekdays can also explain small changes from day to day. The forecast shows an expected amount, so it is usually smoother than noisy daily sales.

The current sweets example has about eight units a day during the Diwali preparation window and about four on nearby ordinary days. Tea and coffee show a winter rise in this demo. These are learned from generated examples and should not be treated as universal rules for every shop.

A worked order example

A supplier takes four days to deliver. You check stock every seven days. The app therefore plans for eleven days. At 20 units a day, demand is 220. Add a 30-unit safety buffer. Subtract 80 usable units already held and 20 confirmed incoming units. The starting suggestion is 150 units.

Expiry, storage and supplier rules are then checked. If a rule lowers that number, read the reason beside it. A zero order might mean enough stock, unsafe extra stock, an unknown expiry date or too little history. Zero does not always mean "nothing needs attention."

A worked expiry example

There are 80 yogurt units that can be sold over the next five days. If expected sales are six each day, about 30 may sell and 50 may remain at risk. The earliest-expiring stock is used first. This is FEFO, short for First Expiry, First Out.

The model never decides the official expiry date. That date must come from a real batch record. Missing expiry needs checking. Stock that expires beyond the forecast period cannot be declared safe simply because a risk value is blank.

What happens when you upload real data?

All six related tables must pass validation before the app replaces its current dataset. Validation means checking that required columns, dates, IDs and quantities make sense. Failed uploads leave the previous data active.

A new uploaded dataset starts with the simple 28-day average. It does not inherit the demo's model score. Run that dataset's evaluation to compare methods for it. Records are saved to the local database only when you choose to save them.

Five-minute presentation

1. Say that this is a synthetic-data demo and show the dashboard.

2. Choose packaged sweets in Product Demand Forecasting. Explain the recorded line, forecast line and shaded festival dates.

3. Explain the model name and why the project chose it. Use MAE as the main error measure; do not claim an accuracy percentage.

4. Open the reorder explanation and work through the 150-unit example above.

5. Show an expiry warning and the new sugar-free product with insufficient history. End with the need for real shop data.

Terms translated into everyday language

Term	Simple meaning
SKU / product ID	A code for one product, such as P11 for packaged sweets
Forecast horizon	How many future days are predicted
Feature	An input the model uses, such as weekday or time of year
Training	Letting the model learn patterns from past records
Validation period	An earlier test period used to choose a method
Test period	A later period used to report the chosen method's errors
Baseline	A simple method used for comparison
Stockout	A product ran out, so sales may be below customer demand
Snapshot	The records as of one date; the demo is dated 16 October 2026
Lead time	Days between ordering and receiving stock
Batch	A group of stock with its own manufacturing and expiry details
MOQ	The supplier's smallest allowed order
Backorders	Customer orders still waiting to be supplied
Bias	Whether forecasts tend to be too high or too low on average
Synthetic	Created by a program instead of collected from real business activity
API	A way for two programs to exchange information; a live Revio connection is not implemented here

Part II - What each file contains

File types first

A `.py` file contains Python code. A `.csv` file contains a table. A `.json` file stores named values and lists. A `.md` file is editable written documentation. A `.pdf` file is the formatted version to read or share. A `.png` file is an image. You do not need to understand every line of code to understand how the project fits together.

Reports: which one should I open?

File in reports/	What it contains and why it exists
AI_Opportunity_Report.pdf	The shop problem, intended users, relation to Cynbit, possible benefits and limits. Start here for the business idea.
Technical_Approach_Report.pdf	The model name and settings, data specification, error results and stock calculations. Read this to understand how it works.
Complete_Project_Explanation.pdf	The everyday-language walkthrough, this file guide and short interview answers. Use it while learning or demonstrating the app.
Matching .md files	Editable text for the three PDFs. The Complete Project Explanation is assembled from the guide files listed below.

Data files: the six tables

All six CSVs are in data/sample/. They are linked by product_id, so a sale for P11 can be matched to the correct product and stock. The row counts below describe the bundled demo, not a required size for uploads.

File	What one row means
products.csv: 13 rows	One product and its price, supplier and ordering rules
sales.csv: 12,240 rows	One product's sales and stock movement on one date
batches.csv: 24 rows	One current group of stock with its own expiry information
purchase_orders.csv: 3 rows	One supplier order and its expected arrival date
reviews.csv: 14 rows	One example customer review
festivals.csv: 28 rows	One named festival on one date, with its source

products.csv columns

product_id is the code used to connect files. product_name and category describe the item. selling_price is what the customer pays; cost_price is the example purchase cost. supplier_id identifies the supplier.

lead_time_days is the delivery wait. minimum_order_quantity is the smallest supplier order. pack_size is the number of units in a pack. shelf_life_days is the assumed remaining life of a new delivery, not the official expiry date of existing stock. max_stock is the storage limit in units. backorders are orders still waiting to be supplied. aliases are other names used to match a product in reviews.

sales.csv columns

date and product_id identify the day and product. quantity_sold is the recorded sale and unit_price is the price per unit. These four fields are required for uploaded sales.

The demo also records opening_stock, received_units, available_stock and closing_stock. They explain what was on the shelf, what arrived and what remained. stockout marks a known shortage. prior_stock_writeoff records carried stock removed before the opening balance used that day. These fields help explain why a low sale may not mean low demand.

promotion marks the example promotional days generated in the demo. It is not currently used as a future promotion plan by the selected model. simulated_demand is the generator's demand before stock limits. It is only used for a separate evaluation and must not be confused with a real observable sales field.

batches.csv columns

batch_id identifies a batch and product_id links it to the catalog. manufacturing_date and expiry_date are recorded dates. quantity_remaining is the amount still held. cost_price and supplier_id give the cost and supplier. Blank batch dates mean unknown. The app must not invent them.

purchase_orders.csv columns

purchase_order_id identifies the order. product_id and quantity_ordered say what is coming. order_date says when it was ordered and expected_delivery_date says when it should arrive. status is confirmed, draft, received or cancelled. Only relevant confirmed future deliveries count as incoming supply in planning.

reviews.csv columns

review_id identifies a review. date, rating and review_text contain the feedback. product_id can identify a product if it is already known; otherwise it can be blank. The simple word rules try to find product mentions and concerns. The reviews are written examples, not a collected customer dataset.

festivals.csv columns

festival_name, festival_date and region describe the event. calendar_source says where it came from. The bundled file also has source_title, source_url and date_status so you can inspect the original name, link and any tentative label. All-day dates are kept as listed by the calendar.

Other data files

File	Meaning
data/sample/metadata.json	States that the business data is synthetic; records seed 42, the snapshot date, version and table row counts.
data/calendar/google_india_holidays.json	Saved public Google Calendar entries and their source details. It is not a login token or private appointment file.
.local/snapshot.sqlite	Optional local database created when Save snapshot locally is used. It contains the saved tables; it is not the main source of bundled demo data.

Application code

File	What it does
app/app.py	Builds the seven screens, responds to controls and connects the data, forecasts and stock calculations.
src/data.py	Reads the CSVs, checks them, prepares usable sales history and saves or loads the local database.
src/seasonal_model.py	Trains the selected product-by-product seasonal Poisson model and calculates its predictions.
src/forecasting.py	Runs all four forecasting methods, checks whether a product has enough history, compares errors and chooses a method.
src/calendar_data.py	Reads the saved public calendar and makes event names and dates consistent.
src/seasonality.py	Summarizes historical monthly sales and past festival periods. These summaries are not a second forecast.
src/inventory.py	Calculates draft orders, uses earlier-expiring stock first and checks storage, delivery, shelf-life and pack rules.
src/reviews.py	Matches product names and simple English phrases to requests or complaints. No LLM is used.
src/__init__.py	Marks src as a Python package so its modules can be imported.

Scripts you can run

File	What running it does
scripts/generate_data.py	Recreates the synthetic demo tables. It is not a script for collecting real sales.
scripts/evaluate.py	Repeats the time-based model comparison and writes the current error report and prediction table.
scripts/build_reports.py	Converts the report text into PDFs and assembles the complete guide. It does not train a model.

Evaluation files: proof behind the numbers

File	What it contains
evaluation/metrics.json	Model chosen, training/test dates, overall error scores and results by product, season and event window.
evaluation/backtest_predictions.csv	Individual historical predictions, their actual values, model names and periods. Use it to investigate a particular mistake.
evaluation/archive/v1/metrics.json	Old results from the first version. Different data or test periods make direct comparison unsafe.
evaluation/archive/v2/metrics.json	Old results before the latest hot-drink and festival correction. Kept as history, not the current score.

"Backtest" means pretending to stand at an earlier date, predicting later dates, and then comparing with what happened. In the CSV, origin is that earlier date, horizon is how many days ahead the prediction is, prediction is the estimate and actual is the usable recorded sale. split says validation or test. A blank actual can mean the sale was not usable for scoring.

Tests: checks on the program

File	What it checks
tests/test_data.py	Invalid files, data rules and local saving/loading
tests/test_engines.py	Forecast and inventory examples, stockouts, expiry, delivery timing and review behavior
tests/test_seasonal_model.py	Calendar dates, product-specific seasons, festival effects, limited history and prevention of future-sales leakage
tests/test_app.py	Navigation through the seven screens and protection against incomplete uploads

There are 26 passing tests in the latest recorded run. This means those checks pass; it is separate from model accuracy.

Written guides and project records

File	What it explains
README.md	Installation, launch commands, project summary and links to the reports
docs/Project_Walkthrough.md	The plain-English explanation of the model, data, screens and examples
docs/File_Guide.md	This guide to the files and their meanings
docs/Interview_Preparation.md	Short answers to common project questions
docs/Company_Research.md	Dated research sources and what was or was not verified about Cynbit
docs/Data_Card.md	Dataset fields, assumptions and changes made to the synthetic data
docs/Calendar_Integration.md	Where festival dates came from and why the app uses a saved calendar copy
docs/Engineering_Decision_Log.md	Historical reasons for technical choices, including old versions. It is a developer record, not the beginner reading guide.
docs/PROJECT_STATE.md	Current project status and where work stands
docs/Execution_Log.md	A dated record of work performed
docs/Development_Log.md	Development milestones and earlier commit references
docs/QA_Record.md	Tests, browser checks and document checks that were performed

Setup, images and working folders

File or folder	Meaning
requirements.txt	Exact Python package versions used by the app and tests
requirements-reports.txt	Extra packages needed to create and inspect PDFs
AGENTS.md	Project workflow rules for the coding assistant
.gitignore	Lists local/generated files that Git should not track
.git/	Git's saved change history and repository information
.venv/	The local Python environment and installed libraries; not application source code
assets/dashboard.png	Screenshot used to show the dashboard
assets/forecast.png	Screenshot used to show the forecast chart
tmp/pdfs/	Temporary rendered pages used to inspect PDF layout; not final reports
.DS_Store	A macOS folder display file; unrelated to forecasting and left untouched

Where to look when something is wrong

If a CSV fails, start with its column names and the error shown by data.py. If a forecast seems wrong, inspect the product's sales history and the saved predictions before changing the model. If the forecast looks reasonable but the order does not, check inventory.py and the batch, supplier and delivery records. If a chart or button fails, start with app/app.py.

Part III - Interview preparation

A short explanation you can say aloud

StockSense is a demo that helps a shop owner review future demand, stock orders and expiry. It uses Seasonal Poisson Regression for the current sample data and compares it with simpler methods. Its average test error is about 2.86 units per product per day, but the data is generated, so real-shop accuracy is still unknown. The stock rules explain each suggested order, and the owner makes the decision.

Questions and answers

1. What does this project do?

It helps a shop owner review demand, stock orders, expiry risks and customer requests.

2. Is it an existing Cynbit product?

No. It is an assessment proposal. A live Revio connection is not implemented.

3. Does Revio already have these features?

We have not verified whether an equivalent inventory feature exists privately. Confirm this with Cynbit.

4. Who would use it?

The proposed users are kirana owners and staff who buy stock. They are not verified existing Revio users.

5. Where did the data come from?

A Python generator created the shop records. Public Google Calendar supplied the festival dates.

6. How large is the dataset?

13 products, 12 with sales over 1,020 days: 12,240 sales rows, 24 batches, 3 purchase orders, 14 example reviews and 28 festival entries.

7. What dates do the sales cover?

1 January 2024 through 16 October 2026.

8. Which model is selected?

Seasonal Poisson Regression, using scikit-learn PoissonRegressor. It trains separately for each eligible product.

9. Why this model?

It predicts nonnegative counts, learns seasonal patterns and had the lowest model-selection error among the tested methods.

10. What does it learn from?

Time of year, weekday, gradual trend and festival preparation windows, fitted against usable past sales.

11. Are seasons a separate prediction?

No. They contribute to the same daily forecast used by stock planning.

12. How are festivals handled?

The week before each festival and the festival day are marked. Past event windows are combined to learn one festival effect per product.

13. Why combine different festivals?

There are too few examples of each named festival to fit reliable separate effects. This is simpler but cannot distinguish each festival independently.

14. What does MAE 2.860 mean?

The average absolute mistake was about 2.86 units per evaluated product per day in the synthetic test.

15. Can I say the model is 81.48% accurate?

No. WAPE is 18.520%, but subtracting that from 100 is not a valid percent-correct accuracy claim.

16. What are RMSE and bias?

RMSE gives more weight to big errors. Bias shows whether forecasts are generally too high or too low. Current values are 3.921 and -0.040 units.

17. How was the model tested?

Four earlier 30-day periods chose the method. Twelve later 30-day periods measured its errors, with training restricted to records available before each forecast.

18. What is a product-day?

One product on one date. The main test scores 4,200 such records.

19. Why not randomly mix dates for testing?

That can let future information help predict the past. Time-ordered tests are closer to how forecasting is used.

20. What are the comparison methods?

A 28-day average, repeating the last week, and HistGradientBoosting, which combines small decision trees.

21. Is the data realistic enough?

It is useful for demonstrating code, but too simplified to establish real-shop performance. We need real records and separate future tests.

22. Why ignore some stockout days?

If a shop sells its last three packets, true demand may be higher than three. That sale does not reveal full demand.

23. What if there is no history?

The app withholds an exact forecast. With some history but not enough for annual patterns, it can use the simple average.

24. What is review period?

The number of days between checking stock and deciding on orders.

25. What is safety buffer?

Extra demand-days of stock kept as a precaution. At 20 units/day, a 1.5-day buffer adds 30 units.

26. What is lead time?

The supplier delivery wait after placing an order.

27. How is an order calculated?

Expected demand during lead time plus review period, plus safety stock, minus usable stock and relevant confirmed deliveries, then adjusted for expiry and constraints.

28. Why might the app suggest less than the target?

Storage, expiry risk, packs or the minimum supplier order can make the full target unsuitable.

29. What is FEFO?

First Expiry, First Out: use the batch that expires earliest before later-expiring stock.

30. Does AI predict the expiry date?

No. The recorded date determines expiry. Forecasts estimate what may remain unsold before it.

31. What does a blank expiry-risk value mean?

The risk may be unknown, such as when expiry is beyond the 30-day forecast. Blank is not a guarantee of safety.

32. Do reviews change demand quantities?

No. Simple word rules highlight issues for a person to investigate.

33. Is an LLM being used?

No. Forecasting uses statistical/ML methods and review checks use English word rules.

34. Is Google Calendar live?

The dates were imported through the connected calendar. The app reads a saved snapshot and does not continuously sync.

35. Why upload all six tables together?

They refer to one another. Checking them together prevents an incomplete or inconsistent replacement.

36. Where are uploaded records saved?

They stay in the local app session unless the user chooses local saving. The saved snapshot uses SQLite.

37. What tests passed?

26 code checks covering data, forecasts, stock decisions, events, seasons and screen navigation. Passing code tests is not proof of real accuracy.

38. Can the app place an order?

No. It creates a draft recommendation for owner review.

39. What would you do next?

Collect permissioned real data, check stock records, compare models on later unseen periods, and measure stockout and expiry costs with owner review.

40. What should I avoid claiming?

Do not call generated data Cynbit data, claim an unverified integration or missing feature, present WAPE as accuracy, or claim measured real-store savings.